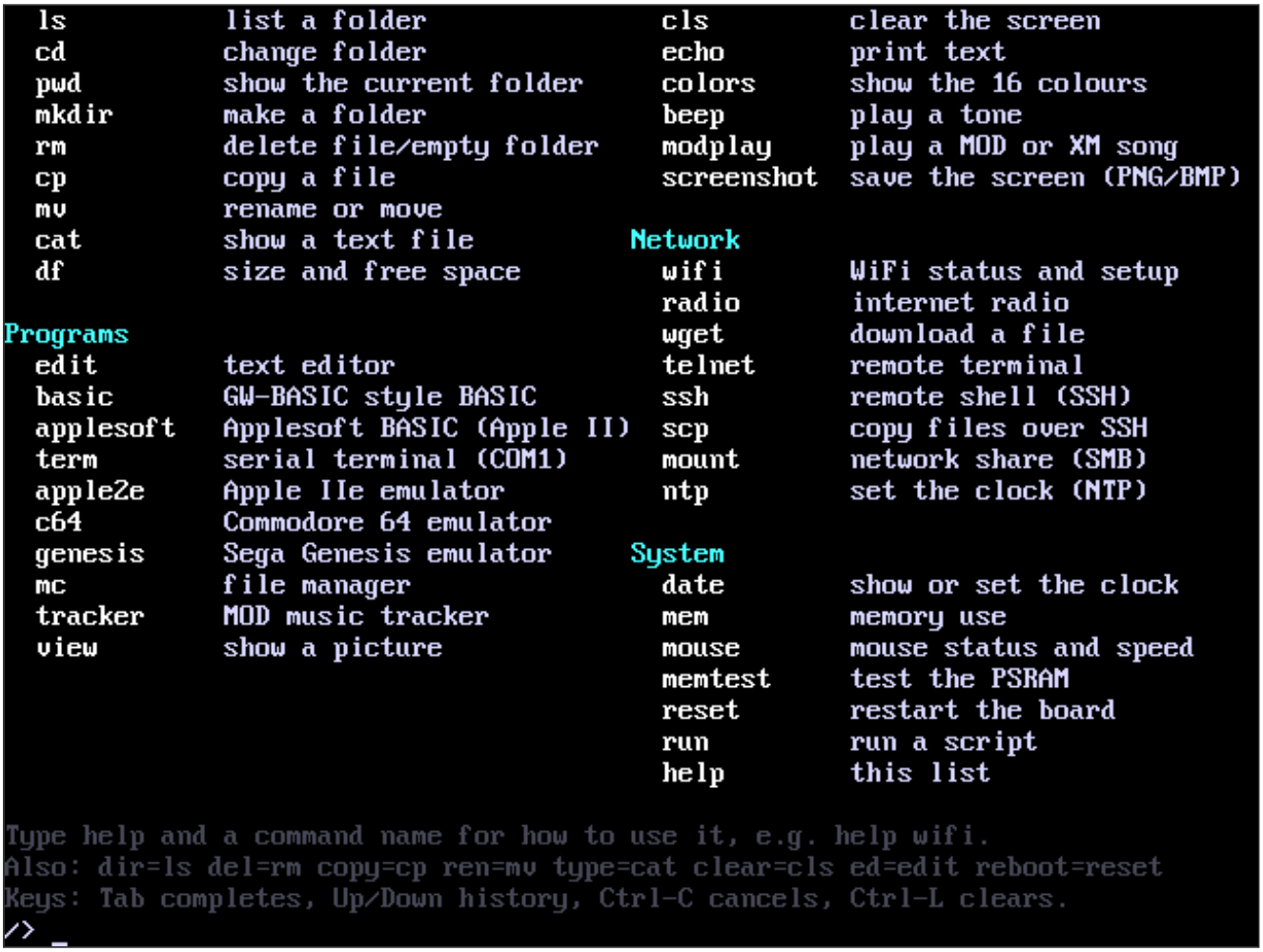
Pico 2 mini OS User's Manual

Pico 2 mini OS · October 27, 2026



Contents

1. Getting started
   /AUTORUN
2. The command line
   Editing keys
   Paths
   help
3. Commands
   Files and folders
   Programs
   Screen and sound
   System
   Network
4. WiFi, network shares and time
   WiFi
   Network shares (mount)
   Date and time
5. Files and folders the OS uses
6. The console
   Screenshots and pictures
7. The mouse
8. Sound
9. Messages
10. Hardware
   Pins
   Clocks
   Memory
11. Building and flashing
12. The manuals

Pico 2 mini OS is a small command-line operating system for the Pimoroni Pico Plus 2 (RP2350B). It shows an 80 x 30 text console on HDMI, takes input from a USB keyboard and mouse, keeps files on a USB flash drive, and connects to WiFi through an RM2 breakout.

This manual covers the system itself: starting up, the shell and its commands, files, WiFi and network shares, the console, the mouse and sound, the hardware and memory, and building. Each application has its own manual (the list is in section 12).

1. Getting started

You need:

- a Pimoroni Pico Plus 2 with 8 MB PSRAM,
- an HDMI monitor, connected through a Pico DVI Sock (GP12-GP19),
- a USB hub on the USB port, with a USB keyboard and a FAT32 flash drive plugged into it,
- optionally, an RM2 WiFi breakout on GP32-GP36,
- optionally, a 3.3 V serial adapter on GP0 (TX) / GP1 (RX) at 115200 baud.

At power-on the screen shows a title bar with the CPU clock and PSRAM size:

```
Pico 2 mini OS                          RP2350 @ 252 MHz, 8 MB PSRAM
```

Below it are status lines for the keyboard, the drive (size and free space), WiFi and the mouse ("none" if no USB mouse is plugged in). These lines update in place while each device comes up. The prompt appears once the keyboard and drive are ready. If no drive shows up within 10 seconds, the prompt appears anyway. Pressing a key on the serial console also brings up the prompt.

About 2.5 seconds after power-on (once the monitor has unmuted its sound) a short chime plays: a soft C major arpeggio of bell tones, made as it plays by `src/chime.c`.

/AUTORUN

If the drive has a file `/AUTORUN` (no extension), its lines run at start-up, once the keyboard and drive are ready, as if typed at the prompt. Lines starting with `#`, `REM` or `::` are comments; `@` in front of a command (or a line `echo off`) stops it being shown; a command that fails doesn't stop the rest. Press Esc during start-up to skip it, or while it runs to stop it. `run FILE` (or `source FILE`) runs any such script by hand. At start-up nobody is there to type, so `mount` uses the login saved in `/SMB/CREDS` (mount the share once by hand and choose to save the login) and otherwise says so and goes on; `wifi wait [seconds]` waits for WiFi first. For example:

```
# mount the PC's share at start-up
wifi wait 20
mount //mypc/share /NET
cd /NET
```

`examples/AUTORUN` in the source has this with comments.

The prompt shows the current folder:

```
/> ls
```

Everything printed also goes to the serial console, and the serial console can type commands. It works as a full VT100 terminal.

2. The command line

Editing keys

Key	Action
Left / Right	Move along the line
Home / End	Start / end of the line
Backspace / Delete	Delete before / at the cursor
Up / Down	Step through the last 16 lines typed
Tab	Complete a command or file name; press twice to list the choices
Ctrl-C	Drop the line
Ctrl-L	Clear the screen and redraw the line
Ctrl-G	Beep
Ctrl-Alt-Del	Restart the board (USB keyboard)

- A line can be up to 255 characters.
- Put quotes around names with spaces: `cat "my notes.txt"`.
- Text pasted into the serial console is buffered (4 KB), so long pastes do not lose characters.

Paths

- Paths use `/`. A path that starts with `/` is from the drive root; anything else is from the current folder. `..` is the parent folder.
- The drive is FAT32, so names are not case-sensitive.

help

`help` lists the commands in groups, one per line (section titles in cyan, command names in white):

```
Files                                 Screen and sound
  ls          list a folder             cls         clear the screen
  cd          change folder             echo        print text
  pwd         show the current folder   colors      show the 16 colours
  mkdir       make a folder             beep        play a tone
  rm          delete file/empty folder  modplay     play a MOD or XM song
  cp          copy a file               screenshot  save the screen (PNG/BMP)
  mv          rename or move
  cat         show a text file        Network
  df          size and free space       wifi        WiFi status and setup
                                        radio       internet radio
Programs                                wget        download a file
  edit        text editor               telnet      remote terminal
  basic       GW-BASIC style BASIC      ssh         remote shell (SSH)
  applesoft   Applesoft BASIC (Apple II)scp         copy files over SSH
  term        serial terminal (COM1)    mount       network share (SMB)
  apple2e     Apple IIe emulator        ntp         set the clock (NTP)
  c64         Commodore 64 emulator
  genesis     Sega Genesis emulator   System
  mc          file manager              date        show or set the clock
  tracker     MOD music tracker         mem         memory use
  si5351      Si5351A clock generator   mouse       mouse status and speed
  view        show a picture            memtest     test the PSRAM
                                        reset       restart the board
                                        run         run a script
                                        help        this list

Type help and a command name for how to use it, e.g. help wifi.
Also: dir=ls del=rm copy=cp ren=mv type=cat clear=cls ed=edit reboot=reset
Keys: Tab completes, Up/Down history, Ctrl-C cancels, Ctrl-L clears.
```

`help` and a command name shows how to use that command, e.g.:

```
/> help wifi
wifi - WiFi status and setup

wifi                      status and IP address
wifi scan                 list networks
wifi connect SSID [pw]    connect and save to WIFI.CFG
wifi off | on             radio off / on
wifi forget               delete WIFI.CFG
```

Other names for a command work too (`help dir` shows `ls`).

3. Commands

Files and folders

Command	Alias	What it does
`ls [dir]` or `ls [dir/]pattern`	`dir`	Lists a folder: names with sizes, `<DIR>` for folders, then a count of files, folders and bytes. Hidden and system files are left out. Pauses each page with `-- more (any key, q to stop) --`.
`cd [dir]`		Changes folder. With no argument, goes to `/`.
`pwd`		Shows the current folder.
`mkdir dir`		Makes a folder.
`rm [-r] [-f] path...`	`del`	Deletes files or empty folders. With wildcards it asks `Delete N files? (y/N)` first. `-r` deletes a folder and everything in it, after asking `Delete folder X and everything in it? (y/N)`. `-f` doesn't ask. It won't delete the whole drive or the current folder.
`mv from... to`	`ren`	Renames, or moves files into the folder `to` (on the same drive; between the drive and a share, copy then delete).
`cp [-r] [-y] from... to`	`copy`	Copies files. Into a folder they keep their names; several files need a folder. `-r` copies folders with everything in them: `cp -r A B` makes `B` a copy of `A`, or `B/A` if folder `B` already exists; a folder can't be copied into itself. Copies of several files ask before overwriting (`y/N/a` for all), unless `-y`.
`cat file`	`type`	Shows a text file (see below).
`df`		Shows the drive, e.g. `FAT32 "LABEL": 7632 MB total, 7590 MB free, 4 KB clusters`.

Wildcards: `cp`, `mv`, `rm` and `ls` take `*` (any letters) and `?` (one letter) in the last part of a name, in any case, as in DOS: `cp EXAMPLES/*.BAS /NET`, `rm OLD/*.TMP`, `ls *.TXT`. `*.*` also matches names without a dot. A pattern that matches nothing says `No files match ...`. Hidden and system files are never matched, and `cp` and `rm` skip folders that match (`mv` moves them). Each file is shown as it goes, then a count; Esc or Ctrl-C stops between files. Copying a file onto itself is refused.

`cat` details:

- CR characters are skipped, so DOS and Unix text both show correctly.
- VT100/ANSI escape codes pass through, so text with colour codes shows in colour.
- It pages like `ls`. Paging stops once the file moves the cursor itself, as ANSI art does.
- It stops at Ctrl-Z (0x1A), the old DOS end-of-file mark.

Programs

Command	Alias	What it does
`edit [file]`	`ed`	Full-screen text editor (see the *Text Editor* manual).

Command	Alias	What it does
`basic [file]`		GW-BASIC style BASIC; with a file, loads and runs it. See the *BASIC User's Manual* manual.
`view file`		Shows a BMP, PNG or JPEG on the screen, with its size and the load and draw times. Press a key to return.
`mc [left] [right]`		File manager in the style of Midnight Commander (see the *File Manager* manual).
`tracker [file.mod]`	`mod`	MOD music tracker in graphics (see the *MOD Tracker* manual).
`si5351 [set N FREQ | on N | off N | status ...]`	`clockgen`	A Si5351A clock generator on I2C: a graphical panel, or settings from the shell (see the *Clock Generator* manual).
`term [baud] [format]`	`serial`	Serial terminal on COM1 with a settings page (see the *Serial Terminal* manual).
`applesoft [file]`		An Apple IIe at the Applesoft BASIC prompt (see the *Emulators* manual).
`apple2e [disk1 [disk2]]`		An enhanced Apple IIe running its own ROMs, with disk drives (see the *Emulators* manual).
`c64 [file]`		A Commodore 64 running its own ROMs (see the *Emulators* manual).
`genesis [rom]`		A Sega Genesis / Mega Drive with a game list (see the *Emulators* manual).

Screen and sound

Command	Alias	What it does
`cls`	`clear`	Clears the screen.
`echo text`		Prints the text.
`colors`		Shows the 8 colours, their bright versions, bold and reverse.
`beep [hz]`		Plays a 300 ms tone, 20-20000 Hz (default 880 Hz).
`screenshot [file]`		Saves the screen as a PNG or BMP (by the name; default the next free `/SCREENSHOTS/SHOTnnnn.PNG`).
`modplay file`		Plays a MOD or XM (FastTracker 2) song once through HDMI audio, showing the position, tempo, voices and CPU load; any key stops.

System

Command	Alias	What it does
`mem`		Shows memory use (section 10).
`run FILE`	`source`	Runs the lines of FILE as commands (see /AUTORUN in section 1).
`mouse [speed 1-10 | accel on|off]`		Shows the mouse; sets the pointer speed and acceleration, saved in `/MOUSE.CFG` (section 7).
`memtest [KB]`		Writes and checks PSRAM patterns (default 1024 KB).

Command	Alias	What it does
`memtest soak [minutes] [scrub-seconds]`		Runs a long background PSRAM test (default 10 minutes).
`memtest soak stop`		Stops the soak test.
`date`		Shows the date and time (section 4).
`date YYYY-MM-DD HH:MM[:SS]`		Sets the clock by hand (UTC).
`ntp [server]`		Gets the time from an NTP server now and sets the clock (section 4).
`reset`	`reboot`	Restarts the board.
`reset bootsel`		Restarts into the RP2350 USB bootloader, for copying a `.uf2` file.
`help [command]`		Lists the commands, or shows how to use one.

Network

Command	What it does
`wifi`	Shows the WiFi status and IP address.
`wifi scan`	Lists networks in range.
`wifi connect SSID [password]`	Connects and saves the settings in `WIFI.CFG`.
`wifi off` (or `disconnect`) / `wifi on`	Turns the radio off / on.
`wifi wait [seconds]`	Waits until WiFi is connected (default 20 s): for /AUTORUN.
`wifi forget`	Deletes `WIFI.CFG`.
`wget [-k] URL [file]`	Downloads a file over http or https.
`telnet host [port]`	Telnet terminal (default port 23); **Ctrl-]** quits. `telnet` alone: a directory of BBSes to pick from.
`ssh [-p port] [user@]host [command]`	Terminal session on an SSH server, or one command's output.
`mount //server[:port]/share [/NET] [-u [domain\]user] [-g]`	Puts a Windows/Samba share in the file tree (section 4). `mount` alone lists them.
`umount [/NET]`	Takes a share away again.
`scp [-P port] [-r] user@host:path local`	Copies from an SSH server.
`scp [-P port] [-r] local user@host:path`	Copies to an SSH server.

See section 4 for details.

4. WiFi, network shares and time

WiFi

`wifi connect SSID password` connects and saves `WIFI.CFG` in the drive root:

```
ssid=MyNetwork
password=secret
```

The password is stored in plain text, so keep the drive private. At start-up the OS connects to the saved network by itself. The status line changes from `connecting` to `connected` with the IP address, or to `can't connect` with the reason. If the network is lost, it tries again every 30 seconds.

Messages after start-up:

```
[WiFi: connected to NAME, 192.168.1.50]
[WiFi: lost NAME, will retry]
[WiFi: can't connect to NAME: reason]
```

There are no `ping` or `ifconfig` commands; `wifi` shows the IP address.

Network shares (mount)

```
mount //server[:port]/share [/NET] [-u [domain\]user] [-g]
mount
umount [/NET]
```

`mount` connects to a Windows or Samba share (SMB) and puts it in the file tree at `/NET`, or at the folder given (e.g. `/DOCS`). Everything that uses files then works on it unchanged: `ls`, `cd`, `cp`, `mv`, `rm`, `mkdir`, `cat`, `edit`, `view`, BASIC's `LOAD`, `SAVE`, `OPEN`, `FILES` and `KILL`, `term`'s file transfers, `scp` and `wget`. `ls /` shows the mount point as a folder.

- **Logging in:** `-u user` or `-u DOMAIN\user` gives the user; the password is then asked for (not shown). Without `-u`, the user is asked for (Enter alone: guest), unless `/SMB/CREDS` has this share. `-g` logs in as guest. After a successful mount with a typed password, `mount` asks whether to save it in `/SMB/CREDS` (plain text, as `WIFI.CFG`; delete the file to forget it).
- **Protocol:** SMB 2.0.2, 2.1, 3.0, 3.0.2 and 3.1.1, NTLMv2 login, and every message signed (HMAC-SHA256, or AES-CMAC for SMB 3), so servers that require signing work. Not supported: encryption (a share or server that requires it is refused with a message), Kerberos, NetBIOS names (use the server's DNS name or IP address).
- **Files:** reads and writes go 64 KB at a time. Names are shown in code page 437 like the drive's (`?` for characters it hasn't got). Files over 4 GB work. `mv` between the share and the drive isn't possible (copy, then delete).
- **Connection:** if it drops (server restart, WiFi), the next file operation reconnects once by itself; files open at the time give an error. A server that stops answering gives an

error after 30 seconds; Ctrl-C or Esc gives up sooner (after 2 seconds of waiting).
- `mount` alone lists the shares with their SMB version; `umount` (or `umount /NET`) disconnects one. Up to 4 shares, 8 open files on shares.

Date and time

`date` shows e.g. `Sun 2026-10-25 23:00:28 UTC`. Once WiFi is up, the clock is set from `pool.ntp.org` (SNTP) by itself, and kept right from then on. `date YYYY-MM-DD HH:MM[:SS]` sets it by hand; `date` then adds `(set by hand)` until the clock is next set from the network.

`ntp [server]` asks an NTP server for the time straight away and sets the clock. The server is a name or an IP address (default `pool.ntp.org`). It shows the time and how far off the clock was:

```
/> ntp
Time set from pool.ntp.org (74.6.168.72): Sun 2026-10-25 23:20:15 UTC
The clock was 7 ms ahead; round trip 65 ms, stratum 2
```

- It waits up to 5 seconds, asking again every 1.5 seconds; Ctrl-C or Esc stops it.
- `No answer from ...: it may not be a time server` means nothing came back.
- `Unknown host ...` means the name could not be looked up.
- BASIC's `DATE$` and `TIME$` read the same clock.

The time is always UTC; there are no time zones.

5. Files and folders the OS uses

Path	Use
`/WIFI.CFG`	Saved WiFi network (`ssid=` and `password=` lines)
`/CACERT.PEM`	CA certificates for `wget` https
`/SSH/KNOWN_HOSTS`	Trusted SSH server keys
`/SSH/ID_ECDSA`, `/SSH/ID_RSA`	Optional private key for scp login
`/SMB/CREDS`	Saved share logins for `mount` (only if you said yes)
`SCREEN.BMP`	Default `screenshot` file name
`/TERM.CFG`	Saved serial terminal settings
`TERM.LOG`	Default serial terminal log file
`~basic.tmp`	Short-lived scratch file while BASIC's `EDIT` runs; deleted afterwards

6. The console

The console is 80 columns by 30 rows on HDMI (640 x 480). The same output goes to the serial port, so a terminal program there sees the same screen.

For remote sessions (ssh, telnet) it also does some xterm things: window titles and similar strings (`ESC ]`, `ESC P`) are skipped, `ESC [ ? 1049 h` / `l` switch to and from an alternate screen, `ESC [ ? 1 h` puts cursor keys in application mode, `ESC ( 0` and `ESC ) 0` with SO/SI select DEC line drawing, and `ESC [ 6 n` (cursor position) and `ESC [ c` (terminal type) are answered.

It understands these VT100 / ANSI escape sequences (ESC [... letter):

Sequence	Meaning
`A` `B` `C` `D`	Cursor up / down / right / left n
`E` `F`	Cursor to the start of the next / previous line
`G`, `d`	Cursor to column / row
`H`, `f`	Cursor to row;column
`J`, `K`	Erase screen / line (0 to end, 1 to start, 2 all)
`L`, `M`	Insert / delete lines
`@`, `P`, `X`	Insert / delete / erase characters
`S`, `T`	Scroll up / down
`r`	Set the scroll region
`s`, `u`	Save / restore the cursor
`?25h` / `?25l`	Show / hide the cursor
`?7h` / `?7l`	Auto-wrap on / off
`m`	Colours and attributes (below)

ESC 7 and ESC 8 save and restore the cursor and attributes. ESC D, ESC M and ESC E are index, reverse index and next line.

Attributes for `ESC [ ... m`:

Code	Meaning
0	Reset
1 / 22	Bold (bright) on / off
7 / 27	Reverse on / off
30-37, 90-97	Foreground colour, normal / bright
40-47, 100-107	Background colour, normal / bright
39 / 49	Default foreground / background

Colours 0-7 are black, red, green, yellow, blue, magenta, cyan and white. `colors` shows them all.

Screenshots and pictures

Print Screen (or **Ctrl-Alt-S**) takes a screenshot anywhere: the shell, the editor, BASIC's text and graphics screens, mc, the tracker, term, ssh. The picture is what was on the screen when the running program next reads a key (at once, in anything waiting for keys). A box then asks for the file name, filled in with the next free `/SCREENSHOTS/SHOTnnnn.PNG` (the folder is made if needed): edit it (Left/Right, Home/End, Backspace, Ctrl-U clears), Enter saves, Esc cancels. A name ending in `.BMP` saves a BMP instead of a PNG. The keys typed into the box don't reach the program, and the screen is put back as it was. PNGs are 640 x 480 with the 256 RGB332 colours (about 25 KB for a text screen; a BMP is 300 KB). Screens shown line-doubled (the Apple IIe, some BASIC modes) are saved at full height as seen.

`view file` shows a BMP, PNG or JPEG picture on the screen, with its size and how long it took to load and draw; any key returns.

7. The mouse

A USB mouse works plugged into the hub (or a keyboard-and-mouse receiver), any time: the start-up screen lists it, and `[mouse connected]` / `[mouse disconnected]` say when it comes and goes. Mice in the boot protocol work; when the mouse describes a wheel, it is switched to its full reports to get the wheel. Up to four mice move the same pointer.

`mouse` shows it (how many, where the pointer is, the buttons, the speed). `mouse speed 1-10` sets how fast the pointer moves: 5, the default, is half a pixel for each step the mouse reports, 9 is one pixel, 10 a little more. `mouse accel on|off` sets whether fast moves go half as far again (on by default). Both are saved in `/MOUSE.CFG` and read when the drive appears.

- `mc` and the MOD tracker use it (see the *File Manager* manual and the *MOD Tracker* manual).
- BASIC has `MOUSEX`, `MOUSEY`, `MOUSEB` and `MOUSEW`, and `MOUSE ON` / `OFF` for the pointer on the text screen (see the *BASIC User's Manual* manual).
- Programs in C use `src/mouse.h`: the position in pixels, the buttons, a queue of events (move, button down with a double-click flag, button up, wheel), and the pointer: for graphics an arrow that the video output draws over the picture as it sends each line (it never touches the framebuffer, so it can't leave marks or flicker), or for text a reversed cell drawn by the console.

8. Sound

Sound plays through the HDMI monitor's speakers (48 kHz HDMI audio); no extra pin or speaker is needed. The shell has `beep [hz]` and Ctrl-G. BASIC has `BEEP`, `SOUND` and `PLAY`. `modplay` and the tracker play MOD music.

9. Messages

Message	Meaning / what to do
`[keyboard connected]` / `[keyboard disconnected]`	A USB keyboard was plugged in or out.
`[mouse connected]` / `[mouse disconnected]`	A USB mouse was plugged in or out.
`[USB drive: ...]`	A flash drive was mounted (label, size, free space).
`[USB drive removed]`	The drive was unplugged.
`No USB drive. Plug one into the hub.`	The command needs the drive. Plug in a FAT32 drive.
`[USB: the drive can't be reached right now (the hub isn't answering)]`	Unplug and replug the drive or hub.
`[WiFi: ...]`	WiFi status changes (section 4).
`WiFi isn't connected (wifi connect SSID [password])`	A network command needs WiFi first.
`drive full`	No space left; delete some files.
A path followed by a reason (e.g. not found)	A file or folder could not be read or written.
`@@@ WARNING: THE HOST KEY FOR ... HAS CHANGED! @@@`	scp refused a server whose key changed (see the *Network Tools* manual).

10. Hardware

Pins

GPIO	Used for
GP0, GP1	Debug serial port (UART0 TX/RX, 115200 baud)
GP12-GP19	HDMI (HSTX)
GP32-GP36	WiFi breakout (RM2)
GP47	PSRAM chip select
USB port	USB host: hub with keyboard and flash drive

The other GPIOs (of GP0-GP47) are free. BASIC's `PIN`, `PINMODE`, `PWM` and `ADC` refuse the pins above. BASIC's `COM1` serial port can use TX on GP4, 8, 20 or 24 and RX on GP5, 9, 21 or 25. ADC inputs are GP40-GP47.

Clocks

- CPU clock (clk_sys) is 252 MHz, with the core voltage raised to 1.20 V. Building with `-DOS_CLOCK_MHZ=126` gives 126 MHz at the normal voltage.
- HDMI: clk_hstx is 126 MHz, for a 25.2 MHz pixel clock (640 x 480 at 60 Hz).
- Flash runs at clk_sys / 3 (84 MHz); PSRAM runs at up to 42 MHz.

Memory

`mem` prints, for example:

```
SRAM 520 KB: framebuffer 300 KB, heap in use 16 KB, heap free 50 KB
PSRAM 8192 KB: 87 KB in use (6 blocks), 8104 KB free, largest free 8104 KB
```

The figures vary with what is running, connected and mounted. The 8 MB PSRAM holds BASIC's program and variables, the editor's text, images, extra screen pages, the radio's buffers, ROMs and disk images, and network buffers.

The rule: a program holds no SRAM while it isn't running. SRAM (520 KB) is short, so only the system's own parts stay in it all the time; programs take what they need when they start and give it all back when they end. How the SRAM is used:

Part	Size	
Framebuffer	300 KB	640x480, one byte a pixel. The Genesis runs its fastest code from the far end of it (its own screen is smaller).
Shared app area	24.5 KB	One area that the running program has to itself: the C64, the Apple IIe and Applesoft, and the Genesis copy their fast code and variables into it when they start; the radio's decoder uses it while it plays (and gives it up when another of these starts).

Part	Size	
The system	about 95 KB	The network (lwIP's packet buffers 24 KB and heap 8 KB, WiFi driver), video line buffers (12 KB), HDMI sound (9 KB), the console's cells (5 KB), USB, the serial console, the files.
Stacks	24 KB	Each core's 4 KB, and the shell's 16 KB (in the heap).
Heap	the rest	About 50 KB free at the prompt, for programs while they run.

Programs keep anything they must remember between runs (the C64 menu's folder, the Genesis list's place, BASIC's line history) in PSRAM. Buffers that only a program needs (BASIC's variables and stacks, the XM player's voices, the editor's screen copy, the emulators' sound queues, a full-screen program's alternate screen) are made when it starts and freed when it ends.

Below 16 KB of free SRAM heap `mem` adds a warning. Programs that run out of memory say "not enough memory" and carry on (the board doesn't stop with a panic).

For developers: before adding a `static` variable to a program, put it in the program's part of the shared area (`APP_OV_DATA(app)` and `APP_OV_FUNC(app, f)` in `src/app_overlay.h`; the variable is set up again each time the program starts), allocate it when the program starts, or put it in PSRAM if it must last. `arm-none-eabi-nm --size-sort build/pico2_os.elf` shows what is resident.

11. Building and flashing

You need the Pico SDK 2.x, CMake, Ninja and the ARM GCC toolchain.

```
cmake -S . -B build -G Ninja
ninja -C build
```

Build options:

- `-DOS_CLOCK_MHZ=126`: run at 126 MHz instead of 252 MHz.

Flash with a debug probe on the SWD pins:

```
tools/flash.sh              # flashes build/pico2_os.elf
tools/flash.sh other.elf
```

`flash.sh` programs and verifies the image through OpenOCD, clears the debug-enable bit (so the board runs normally) and restarts the board.

Without a probe, type `reset bootsel` (or hold BOOTSEL while plugging in USB). Then copy `build/pico2_os.uf2` to the RP2350 drive that appears on your computer.

12. The manuals

Each manual is a PDF in the `manuals` folder (made from `docs/*.md` with `tools/make_manual.py`):

Manual	File
The core OS: the shell, files, WiFi and shares, the console, hardware	`manuals/Pico 2 mini OS User's Manual.pdf`
Pico BASIC, the GW-BASIC style BASIC (`basic`)	`manuals/Pico 2 mini OS BASIC User's Manual.pdf`
The full-screen text editor (`edit`)	`manuals/Pico 2 mini OS Text Editor.pdf`
The file manager mc, in the style of Midnight Commander	`manuals/Pico 2 mini OS File Manager.pdf`
The MOD/XM music tracker (`tracker`) and `modplay`	`manuals/Pico 2 mini OS MOD Tracker.pdf`
Internet radio, MP3 and AAC (`radio`)	`manuals/Pico 2 mini OS Internet Radio.pdf`
The Si5351A clock generator (`si5351`, `clockgen`)	`manuals/Pico 2 mini OS Clock Generator.pdf`
`wget`, `telnet` and its BBS directory, `ssh`, `scp`	`manuals/Pico 2 mini OS Network Tools.pdf`
The serial terminal (`term`) and XMODEM/YMODEM/ZMODEM	`manuals/Pico 2 mini OS Serial Terminal.pdf`
Applesoft, the Apple IIe, the Commodore 64 and the Sega Genesis	`manuals/Pico 2 mini OS Emulators.pdf`